Image Processing - Exercise 5

Shay Morad

Introduction

In this exercise, we explore GAN-based image reconstruction using StyleGAN2 and Latent Optimization. The goal is to leverage GAN priors to recover high-quality images from degraded inputs, including blurred, grayscale, and masked images. Instead of modifying the generator's weights, we optimize the latent vector (z) to generate images that resemble the given degraded images when processed through the same degradation function.

I use an implemented GAN Inversion, where an image is mapped to its closest latent representation, and use it for image deblurring, colorization, and inpainting. Through this process, we analyze how StyleGAN2 captures natural image priors and how different degradation types impact reconstruction quality. The exercise also provides insight into hyperparameter tuning and the limitations of GAN-generated reconstructions.

Algorithm

The goal of this exercise is to reconstruct degraded images using GAN Inversion and Latent Optimization with StyleGAN2. Instead of modifying the model weights, we optimize the latent vector (z) to generate an image that, when degraded, closely matches the input image.

Step-by-Step Breakdown:

1. **Image Preprocessing & Alignment**
    - Load the input image and preprocess it to match StyleGAN2's expected format (e.g., cropping, resizing to 1024×1024, and aligning faces).
    - If the image is already degraded (e.g., blurred, grayscale, or masked), it is used directly. Otherwise, apply a degradation function.
2. **Latent Code Initialization**
    - Sample a large number (w_avg_samples) of latent vectors z from a standard Gaussian distribution.
    - Pass them through StyleGAN2's mapping network to obtain w-space latent representations (w_avg).
    - Compute the mean latent vector (w_avg) and its standard deviation (w_std).

- Initialize the latent vector w_opt at w_avg, setting it as a trainable parameter.

3. **Optimization Loop ("for num_steps iterations")**
    - Generate Image from Latent Code:
        i. Add small Gaussian noise to w_opt (w_opt + w_noise) for regularization.
        ii. Pass w_opt through the StyleGAN2 generator to generate an image (synth_images).
    - Apply Degradation Function:
        i. If the input image is blurred, apply Gaussian blur to generated image.
        ii. If the input image is grayscale, convert generated image to grayscale
        iii. If the input image is masked (inpainting), apply the same mask to the generated image.
    - Compute Loss:
        i. Extract perceptual features from both the generated image (degraded) and input image using VGG16 feature detector.
        ii. Compute loss as: $\text{loss}=\|\phi(I_{gen})-\phi(I_{target})\|^2+\lambda_{reg}\|w_{opt}-w_{avg}\|$.
        iii. Add to the computed loss the regularization loss to prevent unnatural artifacts by keeping w_opt close to w_avg.
    - Backpropagation & Optimization:
        i. Compute gradients with respect to w_opt and update using Adam optimizer.
4. **Post-Processing & Output**
    - Save intermediate reconstructions periodically to observe progress.
    - After num_steps, extract the final optimized latent code and save it (inverted_latent.npz).
    - Generate the final reconstructed image and compare it to the degraded input.

Results

Results of 3.1

For image aligning, I chose an image of Elon Musk. As can be seen on the left, that is the image before alignment, where Elon's face is not in the center of the image, as well as his whole head being slightly tilted to the left. The resulting image after running the image alignment code is on

the right, where it can be seen that the face is centered and not tilted, as well as the image having smaller dimensions because it is cropped to fit within the GAN's algorithm.



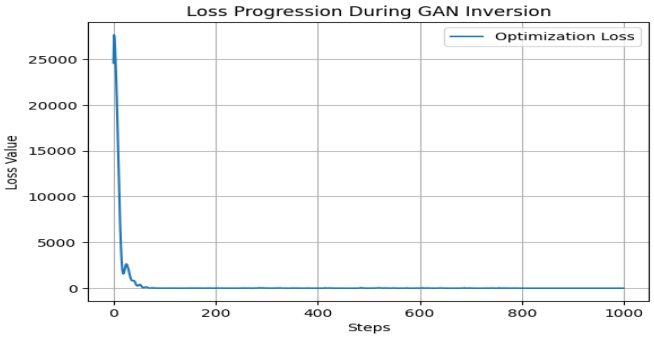
Before Alignment


After Alignment

Results of 3.2

For running the GAN Inversion on a clean input image with no degradation, I chose the aligned image of Elon Musk which can be seen in the results of section 3.1 above. The following images showcase the progression, final result and a loss plot:

Original Image	Initial Generated Image	Step 100 Generated Image
		
Step 300 Generated Image	**Step 500 Generated Image**	**Final Optimized Image**
		



Effect of latent_dist_reg_weight and num_steps on GAN Inversion:

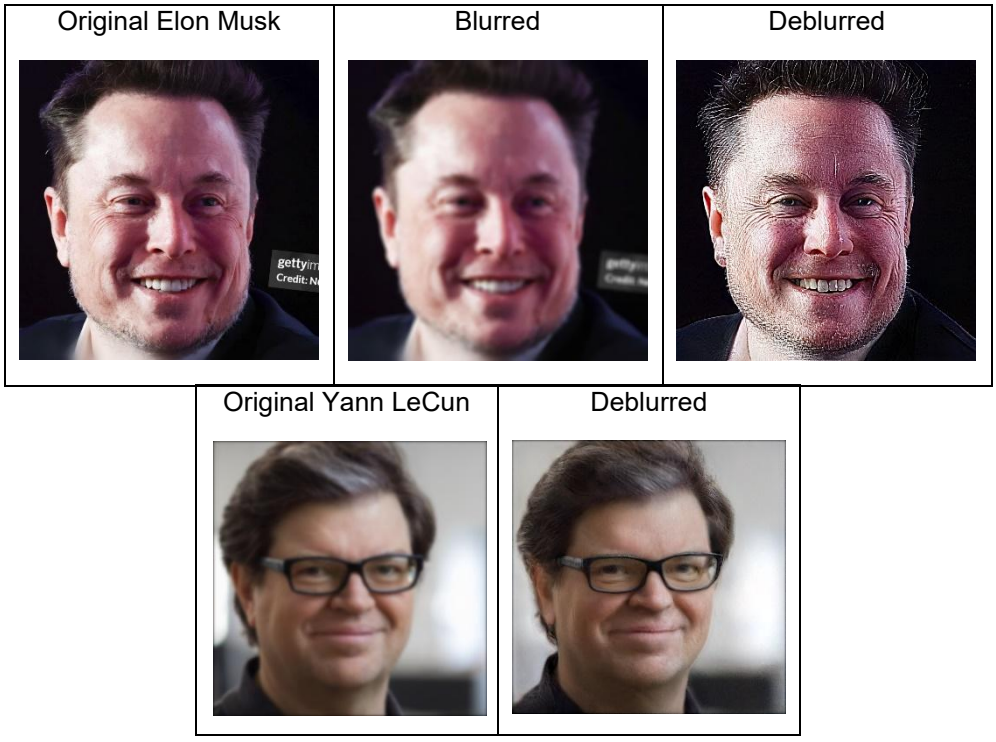
The latent_dist_reg_weight controls how much the optimized latent vector is encouraged to stay close to the average latent space of the GAN. A higher value results in smoother, more natural images but may limit the model's ability to exactly reconstruct the input. A lower value allows for more flexibility, but at the risk of generating unrealistic or distorted images.

The num_steps parameter determines how many optimization iterations are performed. A higher number of steps generally improves reconstruction quality as the latent vector is refined progressively. However, too many steps can lead to overfitting, where the model captures fine details at the cost of unnatural artifacts. A lower number of steps may lead to a faster but less accurate reconstruction.

In practice, balancing these two hyperparameters is crucial. A moderate latent_dist_reg_weight ensures the reconstructed image remains realistic, while a sufficient num_steps ensures enough optimization is done to align with the target image.

Results of 3.3.1

For this section regarding deblurring, the following are the results on the aligned image of Elon Musk (image of my choice) and Yann LeCun (given image):



My solution focused on trial and error, by first finding around what number of iterations each image converges to a final image and then optimizing the latent distance

regularization weight. I also had to test different kernel sizes and sigmas for the gaussian kernel in order to blur the images, and realized that the effect of the kernel size on the image deblurring was that smaller kernel sizes produced barely blurred images, making it easier for the model to reconstruct fine details and be closer to the original image, and as such we don't see the strong effect of the algorithm, while larger kernel sizes introduced strong blurs and as such the deblurring process has a higher chance of generating unrealistic or hallucinated details, and the final image may look more natural but less identical to the original input as the model fills in missing details based on prior. The following are some examples of results for using small/big kernel sizes:

*Notice the blur is not very noticeable in the 5x5 kernel, while very noticeable in the 21x21 kernel


5x5 kernel blur

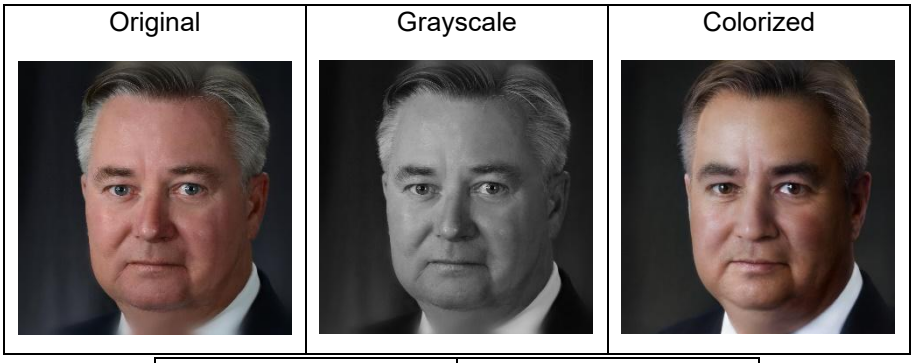

5x5 kernel result

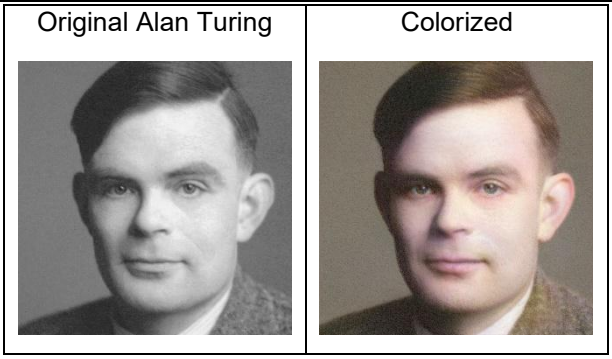

21x21 kernel blur


21x21 kernel result

Results of 3.3.2

The following are the results of the colorization for an image I chose from the FFHQ dataset and the given Alan Turing image:

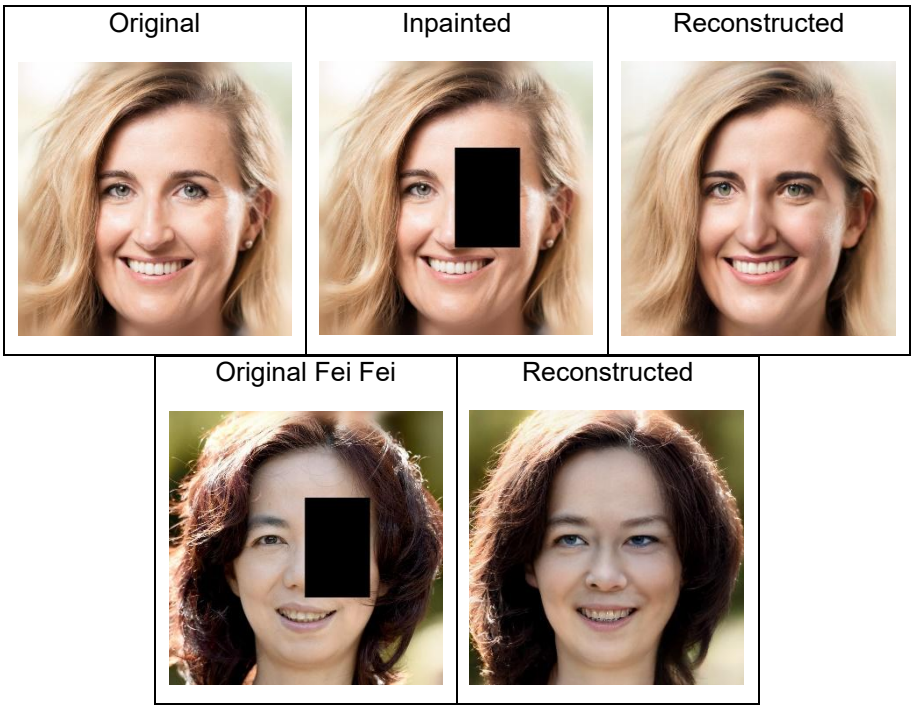




My solution for this section focused on trial and error as in the previous section. I first ran different amount of iterations for each image to see around what amount the run converges, and then continued optimizing by running with different latent distance regularization weight.

Results of 3.3.3

The following are the results of the inpainting for an image I chose from the FFHQ dataset and the given Fei Fei image:



Similar to the previous sections, my solution focused on optimizing the hyper parameters of the amount of iterations and the latent distance regularization weight. For the image I chose I had to choose a rather "heavy" weight for the l.d.r, around 0.8, as lower weights resulted in unrealistic images with bad colors. For the given Fei Fei image, using similar weight resulted in the best results, which is a weight of 0.7 for the l.d.r hyper parameter.

Conclusion

The exercise demonstrated the power of GAN inversion and latent optimization for image reconstruction. By leveraging StyleGAN2's learned priors, we successfully restored degraded images, including blurred, grayscale, and masked inputs. Experimentation with different hyperparameters highlighted their impact on reconstruction fidelity and realism. While small modifications preserved input details, larger adjustments led to more natural but less accurate results. Overall, the approach effectively balances reconstruction accuracy with perceptual quality, showcasing the benefits of deep generative models for image restoration tasks.